

OPTIMAL HASH TABLE SIZE FOR 3D POINT CLOUD CLASSIFICATION

Esranur Aygün

Department of Computer Engineering

Bahçeşehir University

İstanbul, Türkiye

esranur.aygun@bahcesehir.edu.tr

Abstract—3D point cloud classification is a crucial task in modern systems. However, it often needs a lot of memory. This makes it difficult to run on edge devices like the NVIDIA Jetson Nano that operate on only 4 GB of shared memory. To address this, we propose spatial hashing to compress the 3D data. 3D point clouds are converted into a $32 \times 32 \times 32$ binary voxel grid, and only their occupied parts are mapped to a fixed-size 1D memory array using a single-resolution spatial hash function. A lightweight Multi-Layer Perceptron (MLP) classifies the compressed data. This method is tested against a standard 3D Convolutional Neural Network (CNN). The CNN achieved 95.7% accuracy with 4.52 MB memory consumption. In contrast, the hash model with a table size of $T=4096$ reached 85.3% accuracy with just 0.12 MB of memory. This represents a $37.6\times$ memory reduction with only a small drop in accuracy. It is also demonstrated that if memory is compressed too aggressively, down to $T=256$, the hash collision rate exceeds 74% and accuracy drops to 44.2%. These results prove that spatial hashing is a viable technique for 3D perception on memory-limited edge devices.

Index Terms—3D Point Cloud Classification, Spatial Hashing, Edge Computing, Memory Compression, Voxel Grid

I. INTRODUCTION

In recent years, understanding 3D data has become essential, especially for robots, self-driving cars and smart devices. This type of systems rely on point clouds from LiDAR or depth sensors to understand and navigate in their surroundings [1]. They also need to classify 3D shapes and objects in real time.

A recent method called multiresolution hash encoding from Instant-NGP architecture brought a revolutionary perspective to how 3D space was processed by proving neural networks ability to render instant high-quality 3D scenes [2]. However, it was built for high-quality graphics, meaning it consumes 200-500 MB of GPU memory for hash tables. While it would be trivial for a desktop computer but create severe bottlenecks for edge devices like NVIDIA Jetson Nano, which only operate on 4 GB shared memory with no dedicated VRAM [3]. For edge devices like this, memory compression solutions become an operational requirement.

One of the standard approaches to 3D object classification is to split the space into dense cube grids called voxels. In this study, voxel grid size is $32 \times 32 \times 32$ that contain 32,768 cells. But in reality, 3D spaces are usually sparse, and dataset analysis shows only about 2% of the cells occupied by the object. Storing a dense array with sparse data wastes massive amounts of memory, because dense grids memory footprints

grow volumetrically $O(N^3)$, despite objects visible surface grows quadratically $O(N^2)$.

Spatial hashing brings a solution to this inefficiency [4]. Compared to storing entire grid, a hash function in $O(1)$ time maps only the occupied voxels into much smaller, fixed size 1D memory array. This avoids storing empty space and more complex pointer-heavy structures.

Since multiple points may end in same memory slot, resulting in hash collision, this study experiments with the threshold collision rate for a neural network before it can no longer maintain its spatial discriminability.

With a isolated hash table size T as independent variable, hypothesis is that a network would tolerate collision rate below 12% to maintain good accuracy. But if the memory is heavily compressed that it exceeds 70%, it would result in geometric distortions and cause the accuracy to drop. Finding this limit, aim is to observe the optimal memory-accuracy trade-off for 3D classification on edge systems.

To reproduce, the source code is available at: <https://github.com/fukichime/3D-pointcloud-spatial-hash-opt.git>

II. RELATED WORK

A. Neural Graphic Primitives

The recent progress on 3D scene representation is heavily led by neural graphic primitives. Müller et al. introduced a fast multiresolution hash encoding framework known as Instant-NGP [2]. This work showed fast 3D processing by replacing a massive, fully connected later with a small hash table. While Instant-NGP utilize large hash tables up to size 16777216 to achieve realistic results, grid is designed to be task-agnostic [2]. This study builds on this idea by implementing spatial hash mechanism for object classification on limited memory environments.

B. Volumetric Shape Analysis

For evaluating the compression of hash representations, method will be tested on foundational ModelNet40 dataset [5]. Shape of the 3D CAD models are extracted by converting into $32 \times 32 \times 32$ binary voxel grid. This resolution and format are chosen, because Wu et al. showcased in their work that representation above a $30 \times 30 \times 30$ grid gives enough spatial context for deep learning models to learn and classify structures [5].

C. Hashing and Compression

The Core idea of this experiment is a 3D space mapped into a 1D array, which is a technique based on spatial hashing algorithms first developed by Teschner et al. [4]. Since with a limited size memory array, mapping will not be unique, resulting in unavoidable collisions. The expected rate of these collisions follows the classic Birthday Problem [6]:

$$P_{\text{collision}} \approx 1 - \exp\left(-\frac{k(k-1)}{2T}\right)$$

III. METHODOLOGY AND IMPLEMENTATION

A. Pipeline and Data Processing

The data is extracted raw 3D CAD models from a 10-category subset of official ModelNet40 dataset, consisting of the following classes: airplane, car, chair, table, sofa, bed, bookshelf, desk, dresser, and monitor. The raw .off mesh files headers tend to corrupt and crash the library functioning, therefore trimesh library is used to read the geometry and sample 2048 points from each object's surface. This uniform sampling provides enough detail to capture the shape while remaining efficient. After sampling point clouds are centered and normalized to fit in a unit cube, then discretized into $32 \times 32 \times 32$ volumetric grid. The grid is formatted where 1 indicates a point is present and 0 indicates an empty space. Since 3D spaces are vastly sparse and the dataset analysis also proves that only around 2% of the voxels are occupied. Therefore, only the (x, y, z) coordinates of the occupied voxels are extracted to be passed to hash encoded.

However neural networks ability to learn is not inherently ruined by high collision rates. Weinberger et al. established a theoretical proof by showing the exponential tail bounds of interference between these collisions is negligible with high probability [7]. Mainly because hash features inner products are preserved so that MLP can decode the spatial relations even when voxels are collided.

Finally, the definition for a optimal hash table size is derived from model compression literature. Han et al. proved in their framework that neural network storage shrank by 35x to 49x is acceptable trade-off even if it causes small loss in accuracy [8]. Following this, spatial hash grids will be evaluated by finding the optimal memory reduction that keeps classification accuracy within a competitive range of uncompressed dense network.

B. Spatial Hash Encoder

Rather than a massive 3D grid fed into a neural network, voxel coordinated are passed through single-resolution hash function. 3D coordinates are mapped into a 1D array size of T. Teschner et al. has function which relies on specific large prime numbers to distribute them pseudo-randomly :

$$h(x, y, z) = (x \cdot p_1 \oplus y \cdot p_2 \oplus z \cdot p_3) \pmod{T}$$

where $p_1 = 73,856,093$, $p_2 = 19,349,663$, and $p_3 = 83,492,791$ [4]. The hash function calculation was separated from the GPU training loop. To prevent GPU bottlenecks when

resolving hash indices for point clouds, all hash indices were precomputed on CPU during initialization. This allows the GPU to focus on fast memory lookups and reduce training time.

C. Network Architecture

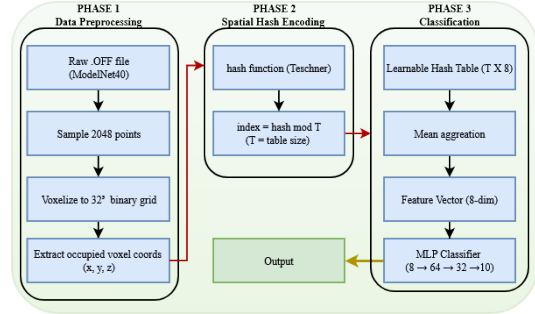


Fig. 1. Architecture of the proposed spatial hash model.

To study how hash table size T effects the performance, a hash model and convolutional neural network architecture (CNN) were compared.

a) *The Spatial Hash Model:* The precomputed hash indices look up feature vector (8-dimensional) from a learnable memory table size T. Since the amount of occupied voxel changes from shape to shape, vectors are aggregated using mean-pooling to get a global 8-dimensional descriptor. This results in a heavily compressed vector which MLP with two hidden layers (64 and 32 neurons) and dropout rate 0.1 that gives the 10-class outputs. The flow of the spatial hash model is illustrated in Fig. 1

b) *3D CNN Baseline:* To compare the performance with collision-free method, uncompressed, dense 32^3 binary grid is used for standard 3D Convolutional Neural Network. The CNN is composed of three 3D convolutional layer with 16, 32 and 64 output channels, each using $3 \times 3 \times 3$ kernel with padding 1, followed by batch normalization, ReLU activation and $2 \times 2 \times 2$ max-pooling. After third pooling, 64-channel feature map is flattened to 4096-dimensional vector. Three fully connected layers with dropout 0.1 produce final 10-class predictions. Since CNN slides filter the dense grid, it keeps the local 3D topology and results in zero collisions.

D. Training

Both models trained on PyTorch framework on NVIDIA RTX 5070 GPU. Adam optimizer at learning rate 0.001 and weight decay 1×10^{-5} used to prevent overfitting [9]. A custom collate function in the data loader handled variable length hash index sequences at a batch size of 32. Models were trained for 10 epochs using a Cross-Entropy Loss function, validation accuracy plateaued by final epoch giving stable comparison metrics.

The memory is computed as table size T x 8 dimensional features x 4 byte floats for the learnable hash table. For the 3D CNN baseline as parameters plus $32^3 \times 4$ bytes for the dense grid baseline.

IV. RESULTS AND ANALYSIS

Evaluation of the spatial hash encoder is done through benchmarking its classification accuracy, memory usage and empirical collision rate against the uncompressed dense 3D CNN baseline. All tests used the 972 sample from the test split of ModelNet40 dataset.

A. Collision rate vs. Memory Trade-off

The dense 3D CNN baseline provides a collision-free upper bound by preserving its 3D topology. It achieved 95.7% validation accuracy but requires 4.52 MB of memory including parameters and input grid. Against the baseline, hash table size T swept logarithmically from 1048576 down to 256. Table I shows the measured memory footprint, best accuracy, and empirical collision rate.

TABLE I
MEMORY, ACCURACY, AND COLLISION RATE COMPARISON.

Method	Mem. (MB)	Acc. (%)	Coll. (%)
3D CNN Baseline	4.52	95.7	0.0
Hash $T = 2^{20}$	32.00	85.8	0.0
Hash $T = 2^{16}$	2.00	82.8	0.5
Hash $T = 2^{12}$	0.12	85.3	11.8
Hash $T = 2^8$	0.01	44.2	74.3

B. Empirical Performance

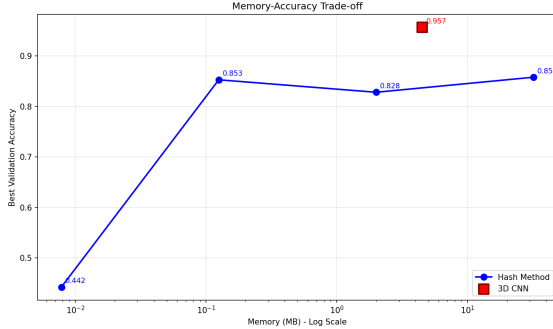


Fig. 2. Memory-Accuracy Trade-off plots log scaled memory against validation accuracy.

The non-linear trade-off between memory footprint and classification performance is illustrated by the Pareto curve in Fig. 2

Oversized tables such as $T = 1048576$ uses 32.00 MB of memory, which is larger than CNN, but yields an accuracy of 85.8%. With zero collisions, expanding the hash space beyond its geometric boundaries provides negligible benefits.

Optimal point occurs at $T = 4096$. This configuration achieved 85.3% accuracy with 0.12 MB. In comparison to CNN, 37.6x memory reduction with only 10.4% accuracy drop. The slight dip at $T = 65,536$ with accuracy of 82.8% is attributed to training variance, both $T = 65,536$ and $T = 4,096$ far outperform the collapse threshold.

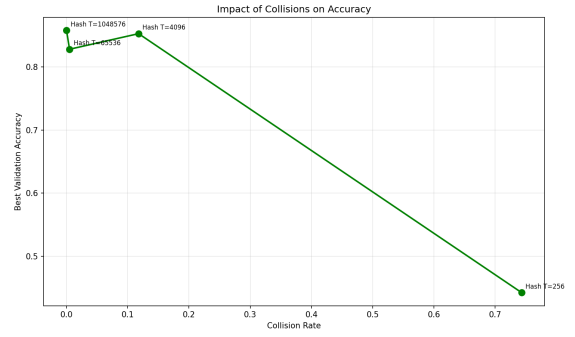


Fig. 3. Impact of Collision on Accuracy.

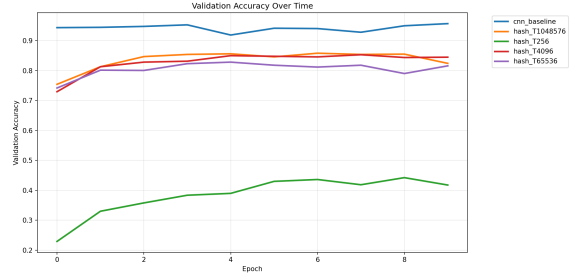


Fig. 4. Validation accuracy over time.

C. Mathematical Collapse

To validate the theoretical limits of the hash encoder, critical threshold analyzed where mathematical interference destroys the spatial information. Fig. 3 isolates the correlation between empirical collision rates and model accuracy, while Fig. 4 tracks the training convergence over 10 epochs.

As shown in Fig. 3, the empirical collision rate of 11.8% at $T=4096$ is tolerated by the network. The MLP performs efficient as spatial filter, despite moderate feature overlap. Furthermore, the training curves in Fig. 4, which show the $T = 4096$ model with the red line and oversized $T = 1048576$ model with orange line converge smoothly and almost identically alongside the CNN baselines blue line.

However, when the table size is severely restricted to $T = 256$ with 0.01 MB memory, the collision rate spikes to 74.3%, mirrors the theoretical expectations derived from the Birthday Problem. Under high collision rates, structural primitives such as wing of an airplane and leg of a chair are mapped to the same slot, which would result in degradation of 44.2% shown in Fig. 3. In convergence data in Fig. 4 further confirms, the $T = 256$ model with green line struggles to learn the entire training process and ends up with inferior accuracy level. This confirms spatial collisions exceeding networks tolerance results in 3D geometry loss.

V. DISCUSSION AND CONCLUSION

The results demonstrate that a single resolution spatial hash encoder offers memory advantage for 3D point cloud classification on edge devices. As reported in Table I, the optimal point at $T = 4096$ reduces the memory footprint

significantly compared to 3D CNN baseline, while maintaining competitive accuracy. The accuracy gap of around 10% is inherent to the hashing approach. the CNN processes the continuous spatial grid using sliding local filters, allowing it to learn geometric primitives. Whereas the hash encoder scrambles coordinates into a 1D array, forcing the compact MLP to reconstruct spatial relationships from scratch.

Interestingly, moderate hash collisions of 11.8% rate at $T = 4096$ act as a natural regularizer, preventing overfitting that can occur oversized tables. These observations validate the core mechanism of the hash encoding introduced in Instant NGP and as established in the original architecture, scaling this methodology to make use of a multiresolution cascade would likely provide the missing spatial hierarchy needed to close the remaining accuracy gap with the CNN baseline [2].

It can be concluded that spatial hashing is highly viable for edge device classification. Future work will explore multiresolution hash grids, double hashing to mitigate aliasing, and direct deployment on edge devices to measure real time latency and power.

REFERENCES

- [1] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, "PointPillars: Fast encoders for object detection from point clouds," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 12697–12705.
- [2] T. Müller, A. Evans, C. Schied, and A. Keller, "Instant neural graphics primitives with a multiresolution hash encoding," *ACM Trans. Graph.*, vol. 41, no. 4, Art. 102, Jul. 2022.
- [3] NVIDIA Corporation, "Jetson Nano module datasheet," 2019. [Online]. Available: <https://developer.nvidia.com/embedded/jetson-nano>
- [4] M. Teschner, B. Heidelberger, M. Müller, D. Pomeranets, and M. H. Gross, "Optimized spatial hashing for collision detection of deformable objects," in *Proc. Vis. Model. Visual. (VMV)*, 2003, pp. 47–54.
- [5] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao, "3D ShapeNets: A deep representation for volumetric shapes," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2015, pp. 1912–1920.
- [6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [7] K. Q. Weinberger, A. Dasgupta, J. Langford, A. J. Smola, and J. Attenberg, "Feature hashing for large scale multitask learning," in *Proc. 26th Annu. Int. Conf. Mach. Learn. (ICML)*, 2009, pp. 1113–1120.
- [8] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016.
- [9] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.